

Problem solving and search

Unit 2 · Intelligent Systems

School of Computer Science · Ciudad Real
Universidad de Castilla-La Mancha

Rubén Juárez Cádiz



What you will be able to do

A framework for reasoning, tracing and building solvers

Formulate	Turn a task into states, actions, transitions, a goal and costs.
------------------	--

Trace	Follow the frontier, selected node and repeated-state decisions.
--------------	--

Compare	Justify completeness, cost optimality, time and memory.
----------------	---

Implement	Connect BFS, UCS, DFS, DLS and IDS to the Loopover lab.
------------------	---

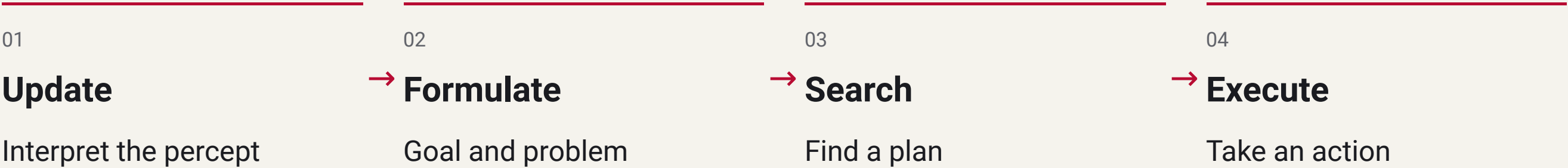
	The central question: which node should leave the frontier next?
--	--

A goal does not specify a route

“Reach the destination” tells us what counts as success.

Search finds an action sequence that can make it happen.

Formulate, search and execute



Mode	What happens
Offline planning	Compute a plan using a model before executing it.
Online search	Interleave action, observation and learning about an initially unknown environment.

A real agent may monitor execution and replan when its assumptions fail.

Different uncertainty, different solutions

Problem type	Information available	Form of solution
Single-state	Known current state and deterministic model.	An action sequence.
Conformant	Initial uncertainty with no observations during execution.	A sequence that works for every possible state.
Contingent	Observations or uncertain action outcomes matter.	A conditional plan or policy.
Exploration	The environment or transition structure is unknown.	Act while acquiring a useful model.

This unit mainly studies deterministic, fully observable single-state problems.

A precise search problem

Component	Meaning
Initial state s_0	Where the search begins.
Actions $A(s)$	Which actions are legal in a state.
Transition $T(s,a)$	Which state results from a legal action.
Goal test $G(s)$	Whether the objective has been reached.
Step cost $c(s,a)$	The cost contributed by one action.

A solution is a legal action sequence. Its path cost is the sum of step costs.

An abstraction must preserve future choices

Useful simplification

Route finding may represent a state by the current city.

Missing information

If fuel determines which roads are usable, city alone is insufficient.

A better state

`(city, fuel)` preserves the information needed for future actions.

Design test

Can identical encoded states have different legal futures?

Discard irrelevant detail while preserving actions, transitions, goals and costs.

Romania: fewest roads or fewest kilometres?

Selected road	Distance
Arad ↔ Sibiu	140 km
Sibiu ↔ Fagaras	99 km
Fagaras ↔ Bucharest	211 km
Sibiu ↔ Rimnicu Vilcea	80 km
Rimnicu Vilcea ↔ Pitesti	97 km
Pitesti ↔ Bucharest	101 km

Three-road route

Arad, Sibiu, Fagaras, Bucharest

450 km

Four-road route

Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest

418 km

Fewer actions and lower total cost are different objectives.

A state graph and its search tree

State-space graph

$S \rightarrow A \rightarrow X$

$S \rightarrow B \rightarrow X$

There is one state X, reached through two routes.

Search tree

$S / A / X$

$S / B / X$

There are two nodes containing X, each with its own path.

States describe configurations. Nodes describe paths that reach configurations.

A node stores more than a state

Field	Example	Purpose
state	X	Configuration used by the problem model.
parent	Node containing A	Reconstruct the selected path.
action	A_to_X	Record how the node was reached.
depth	2	Count actions from the root.
path cost $g(n)$	7	Accumulate step costs.

The same state may occur at different depths and path costs.

Frontier, expansion and reached states

Frontier Candidate nodes waiting to be selected.

Selection Remove a candidate according to the strategy.

Expansion Generate legal successor states and child nodes.

Repeated-state control Track discovery, expansion or best known cost, depending on the algorithm.

Discovered, expanded and settled are different events.

A generic tree-search skeleton

```
frontier ← [root(initial_state)]
while frontier is not empty:
    node ← remove_next(frontier)
    if goal(node.state):
        return reconstruct(node)
    for (action, state, cost) in successors(node.state):
        child ← Node(state, node, action,
                     node.depth + 1, node.g + cost)
        insert(frontier, child)
return FAILURE
```

The removal rule controls search order. Duplicate handling adds another design decision.

Repeated states require a policy

Policy	What it prevents	Important consequence
Path cycle check	A state appearing twice on the current path.	Uses little memory but allows repeats across branches.
Discovered-state set	Adding a state already reached.	Appropriate for ordinary equal-cost BFS.
Best-cost map	Discarding a cheaper route to a known state.	Needed for cost-based search such as UCS.
Depth-aware pruning	Discarding a state reached with more remaining depth.	Relevant when implementing depth-limited graph search.

A boolean “visited” flag is not a substitute for every kind of repeated-state reasoning.

BFS selects the shallowest candidate

FIFO queue

Insert new nodes at the back.
Remove the oldest node at the front.

Expansion order

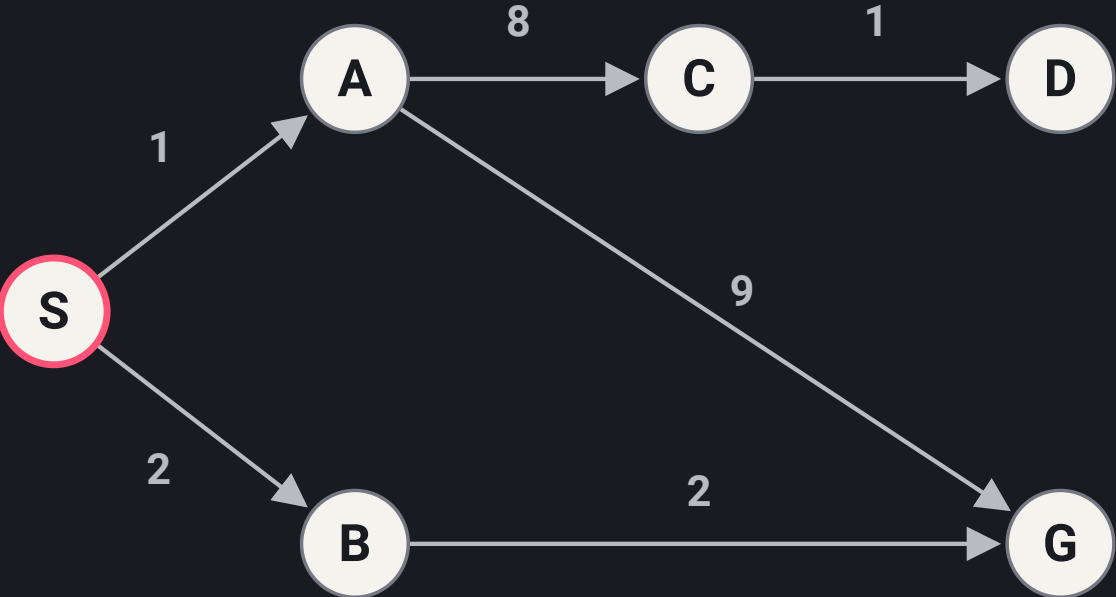
All depth-0 nodes, then depth-1 nodes, then depth-2 nodes, and so on.

What it optimizes

The number of actions. This also minimizes cost when every action costs the same.

The ordering of successors resolves ties between nodes at the same depth.

One graph, three search strategies



Frontier policy

BFS · FIFO

Select next node

Run to completion

Reset

Selection order

None selected

Frontier · next candidate first

S · g=0 · d=0

Ready. S is the initial candidate with $g = 0$.

Directed edges carry positive costs. Successor order is alphabetical. Text labels identify selected and frontier states.

BFS: the queue explains the result

Selected	Queue afterwards	What happened
S	A(1), B(2)	Discover the two depth-1 nodes.
A	B(2), C(9), G(10)	Discover C and G at depth 2.
B	C(9), G(10)	G is already discovered.
C	G(10), D(10)	Discover D at depth 3.
G	Stop	Return S → A → G, cost 10.

Numbers in parentheses are path costs, not queue priorities. BFS prioritizes depth.

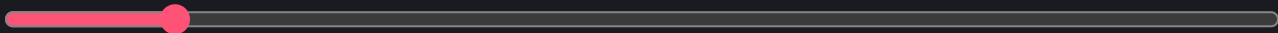
BFS guarantees and memory cost

Question	Answer with assumptions
Complete?	Yes for finite branching when a solution exists at finite depth.
Cost-optimal?	Yes when step costs are equal and positive.
Main resource risk	An exponentially large frontier.
Finite explicit graph	Standard adjacency-list BFS takes $O(V + E)$ time and $O(V)$ space.

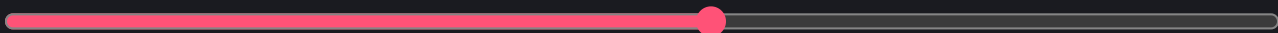
Tree-size bounds and explicit-graph bounds describe different representations.

How quickly does a tree grow?

Branching factor b: 3



Depth d: 6



A full b-ary tree, no repeated states and all levels counted.

Nodes at depth d · b^d

729

Nodes through depth d · $\sum_{i=0}^d b^i$

1,093

At an assumed 128 bytes per retained node

136.63 KiB

The memory estimate is illustrative. Real nodes and data structures have additional overhead.

UCS selects the lowest path cost

Priority queue

$$\text{priority}(n) = g(n)$$

Different objective

A deeper path may still be cheaper.

Different duplicate rule

If a cheaper route reaches a known state, update its best cost and candidate path.

The first discovered route to a state need not be its cheapest route.

Best costs and stale entries

```
best[start] ← 0; push(root, priority=0)
while frontier is not empty:
    n ← pop_min_g()
    if n.g != best[n.state]: continue
    if goal(n.state): return reconstruct(n)
    for (a, s, cost) in successors(n.state):
        new_g ← n.g + cost
        if new_g < best.get(s, infinity):
            best[s] ← new_g
            push(Node(s, n, a, new_g), priority=new_g)
return FAILURE
```

The goal test follows stale-entry rejection and minimum-cost selection.

UCS improves an expensive discovery

Selected	Active candidates, lowest g first	Effect
S(0)	A(1), B(2)	Initialize alternative routes.
A(1)	B(2), C(9), G(10)	An expensive goal path is discovered.
B(2)	G(4), C(9)	Improve G from 10 to 4.
G(4)	Stop	Return $S \rightarrow B \rightarrow G$.

G(10) may remain physically in the heap, but it is stale and must not be accepted.

Discovering a goal is not enough

A discovers G with cost 10.

B later offers G with cost 4.

Stopping on generation

May accept the expensive route before cheaper alternatives are considered.

Stopping on selection

Accepts a non-stale goal removed at minimum g under UCS assumptions.

The position of one goal test can change the correctness of an algorithm.

UCS: separate optimality and completeness

Setting	What can be guaranteed
Finite reachable graph, nonnegative costs	Correct graph UCS terminates and returns a cheapest reachable goal.
Infinite setting, finite branching, $\text{cost} \geq \epsilon > 0$	A finite-cost solution will be reached.
Infinite zero-cost region	Search may never reach a positive-cost goal.
Negative edges	The usual settle-on-removal argument fails.

A zero lower bound and a positive lower bound are not interchangeable.

DFS follows one branch first

LIFO stack

Insert new nodes on top.

Remove the most recently inserted node.

In the shared graph

Selection order:

S, A, C, D, G

Returned path

S → A → G, with cost 10.

The selected-node order is not the solution path.

Low-memory claims need an implementation

DFS variant	Memory behaviour	Completeness caveat
Tree / path-based DFS	$O(bm)$ with stored siblings, or $O(m)$ with lazy successor iteration.	Can descend forever without reaching a goal.
Finite graph DFS with a global visited set	Stores up to $O(V)$ visited states as well as the traversal state.	Complete over a finite reachable graph with correct duplicate handling.
Either version	No general minimum-depth or minimum-cost guarantee.	Successor ordering can strongly affect the first result.

Adding a global visited set changes the memory model.

Depth-limited search has three outcomes

SOLUTION

A goal was found within the limit.

CUTOFF

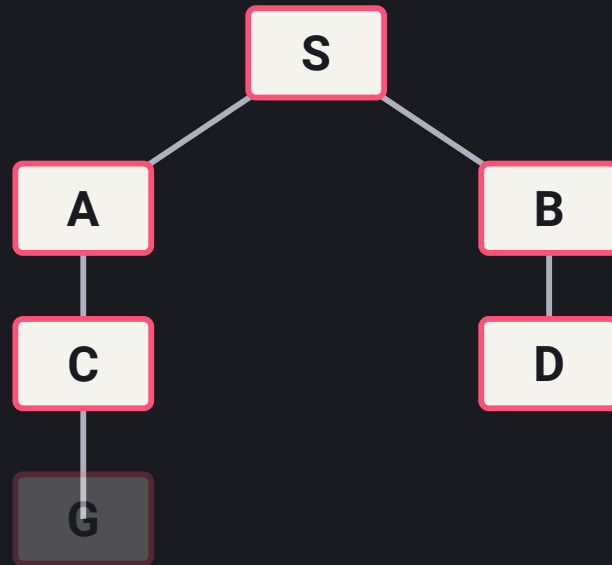
A potentially useful branch was stopped by the limit.

FAILURE

All available branches were exhausted without finding a goal or being cut off.

CUTOFF means “the limit prevented a conclusion”, not “there is no solution”.

A depth limit you can change



Edges: $S \rightarrow A$, $S \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow G$.
G is at depth 3. D is a terminal dead end.

Depth limit ℓ : 2



Result

CUTOFF

Visited nodes in this run

$S \rightarrow A \rightarrow C \rightarrow B \rightarrow D$

At least one nonterminal branch reached the depth bound.

Increasing the limit can change CUTOFF into SOLUTION.

IDS increases the limit systematically

```
for limit = 0, 1, 2, ...:  
    result ← depth_limited_search(initial_state, limit)  
    if result is SOLUTION: return result  
    if result is FAILURE: return FAILURE  
    # CUTOFF: try the next limit
```

01

 $\ell = 0$

Root only

02

→ $\ell = 1$

One action

03

→ $\ell = 2$

Two actions

04

→ $\ell = 3$

Three actions

With finite branching, IDS finds a finite-depth solution without keeping a BFS-sized frontier.

Repeated work can still be affordable

Full tree through depth 5	Single traversal count	IDS total count
b = 2	63	120
b = 10	111,111	123,456

Why the overhead is limited

In a branching tree, most nodes occur near the deepest level.

What the counts assume

Every level is fully explored. No early goal stop and no duplicate pruning.

For $b > 1$, the overhead ratio approaches $b / (b - 1)$ as depth grows.

Completeness and cost optimality

Strategy	Complete?	Cost-optimal?	Main trade-off
BFS	Yes ¹	With equal positive costs	Large frontier.
UCS	Yes ²	With nonnegative costs ³	Cost-ordered frontier and best-cost storage.
DFS	Not in general; yes on finite spaces with safe cycle control	No	Traversal depth versus visited-state memory.
DLS	Only when the limit is sufficient ⁴	No in general	A wrong limit cuts off solutions.
IDS	Yes ¹	With equal positive costs	Repeated shallow work, small path memory.

¹ Finite branching and finite-depth solution. ² Positive lower-bounded costs for the usual infinite-space result. ³ Correct best-cost handling and a goal removed from the queue. ⁴ Finite search to the bound with safe cycle handling.

Notation that keeps the claims precise

Symbol	Meaning
b	Maximum branching factor.
d	Depth of the shallowest goal, not necessarily the cheapest goal.
m	Maximum depth considered by the search tree or traversal.
<i>ℓ</i>	Chosen depth limit.
C*	Cost of an optimal solution.
ε	A positive lower bound on step costs, when assumed.

Do not replace d with “depth of the least-cost solution” when costs differ.

Tree-based worst-case growth

Strategy	Time bound	Space bound
BFS, goal tested on removal	$O(b^{(d+1)})$	$O(b^{(d+1)})$
UCS, cost $\geq \epsilon > 0$	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$
DFS, finite maximum depth m	$O(b^m)$	$O(bm)$
DLS with limit ℓ	$O(b^\ell)$	$O(b\ell)$
IDS, $b > 1$	$O(b^d)$	$O(bd)$

Conventions: finite branching $b > 1$, implicit trees, stored siblings for DFS-style space. A global reached map adds memory. Explicit finite-graph bounds use V and E instead.

Choosing a strategy from the task

Equal costs and ample memory

BFS is a straightforward way to obtain a shortest action sequence.

Different nonnegative costs

UCS handles the actual accumulated cost.

A shallow solution and tight memory

Path-based IDS is often a useful starting point.

A justified maximum depth

DLS makes the bound explicit.

Traversal without an optimality requirement

DFS may be appropriate if its termination conditions are controlled.

Choose the objective and constraints before choosing the data structure.

The 8-puzzle as a state space

Supplied initial board

7	2	4
5	□	6
8	3	1

Goal board

1	2	3
4	5	6
7	8	□

State: tile arrangement. Actions: legal blank moves. Cost: one per move.

The vacuum world: modelling the full state

$s = (\text{location}, \text{dirtA}, \text{dirtB})$

$2 \times 2 \times 2 = 8$ states

Goal: both dirt flags are false.

Actions

Left, right, suck and optionally wait.

Transition

Update the location or current dirt flag.

Cost convention

One per non-wait action. Waiting can be omitted from the search.

This search model uses a known full state. Unit 1's local sensor alone does not provide it.

Limited-inventory coin change

Coin value	Available quantity	Action
50 cents	1	take_50
10 cents	4	take_10
5 cents	1	take_5
1 cent	4	take_1

State = ((n50, n10, n5, n1), remaining). Goal: remaining = 0. Cost: one per coin.

A coin-change solver you can inspect

Target amount Y , in cents

15

Solve with BFS

Available: $1 \times 50c$, $4 \times 10c$,
 $1 \times 5c$ and $4 \times 1c$.

Every selected coin costs one search step.

Minimum number of coins

2

Returned coin sequence

10c + 5c

Expanded states / peak frontier

4 / 4

Verified sum: 15 cents. Inventory limits respected.

The solver uses remaining inventory in state equality and checks the goal after queue removal.

Fifteen cents: a complete short argument

Depth	Selected-coin prefixes	Interpretation
0	[]	Remaining amount 15.
1	[10], [5], [1]	No one-coin solution. 50 is not legal.
2	[10,5] reaches the goal	Two coins suffice.
Optimality	No goal exists at depth 0 or 1	Equal action costs make this a minimum-coin solution.

The [5,10] path reaches the same final inventory as [10,5].

Why greedy coin choice is a separate claim

A different coin system

Values {4, 3, 1}, unlimited supply.

Target: 6.

Largest-first choice

$4 + 1 + 1$ uses three coins.

A better solution

$3 + 3$ uses two coins.

What follows

A plausible local rule is not automatically a proof of global optimality.

This counterexample changes the coin system. It is not a claim about the supplied inventory.

River crossing: compact state and safety

$s = (M_l, C_l, \text{boat})$

Initial: (3, 3, L)

Goal: (0, 0, R)

Right-bank counts follow from:

$M_r = 3 - M_l$, $C_r = 3 - C_l$.

Boat capacity

One or two passengers per crossing.

Safety on each bank

If missionaries are present, they cannot be outnumbered.

$M = 0$ or $M \geq C$

The zero-missionary case matters: a bank containing only cannibals is legal.

Successors must preserve every constraint

01	02	03	04
Load	→ Cross	→ Check	→ Return
Choose one or two available passengers	Update the departure and arrival banks	Validate counts and both banks	Keep only legal successor states
Candidate load (M,C)		Legal passenger count	
(1,0), (0,1)		One	
(2,0), (1,1), (0,2)		Two	

Passenger availability and destination safety are separate checks.

A verified 11-crossing solution

Step	Passengers / direction	Left bank afterwards
1	CC →	(3,1)
2	C ←	(3,2)
3	CC →	(3,0)
4	C ←	(3,1)
5	MM →	(1,1)
6	MC ←	(2,2)

Step	Passengers / direction	Left bank afterwards
7	MM →	(0,2)
8	C ←	(0,3)
9	CC →	(0,1)
10	C ←	(0,2)
11	CC →	(0,0)

→ means left to right. ← means right to left. Every crossing has unit cost.

Geared Loopover connects both units

00	01	02	03
04	05	06	07
08	09	10	11
12	13	14	15

State

The complete 4 × 4 board, immutable as a search key.

Action

Row index, column index and sign.

Successor

Apply row rotation, then column rotation.

Goal and cost

Ordered board, one unit per coupled action.

32 action labels per state: 4 rows × 4 columns × 2 signs.

An exact transition test for 02+

Input

00010203040506070809101112131415

Complete successor

03001402040501070809061112131015

After the row phase

03	00	01	02
04	05	06	07
08	09	10	11
12	13	14	15

03	00	14	02
04	05	01	07
08	09	06	11
12	13	10	15

The column move uses the board produced by the row move.

Theory mapped to Java responsibilities

Course class / structure	Responsibility
Estado	Valid immutable board encoding and state equality.
Sucesor	Action label, resulting Estado and step cost.
Nodo	Estado, parent, action, depth and accumulated cost.
Frontera	Queue, stack or priority queue policy.
Visitados / best-cost map	Repeated-state handling appropriate to the strategy.

Keep the problem model independent of the frontier policy.

A small implementation contract

Immutability

Generating a child must not alter its parent's state.

Equality

Equivalent configurations must compare equal and have consistent hashes.

Cost propagation

$\text{child.g} = \text{parent.g} + \text{successor.cost}$.

Path reconstruction

Follow parent pointers, collect actions and reverse the result.

Deterministic ties

Specify successor order and tie-breaking for reproducible traces.

A search algorithm cannot compensate for an incorrect transition function.

Tests that check the model, not just the output

Check	What a failure reveals
Known transition 02+	Wrong operation order, direction or index.
All 16 tiles preserved	Lost, duplicated or overwritten positions.
Parent unchanged	Aliasing or mutable-state leakage.
Goal input	Incorrect empty-plan handling.
Different-cost graph	A UCS best-cost or goal-test error.
Unreachable finite case	Failure termination or repeated-state handling.

A visually plausible route is not sufficient evidence that the solver is correct.

Workshop: trace, compare and justify

12 min

Use the six-state graph from the search laboratory.

Work in pairs, then compare with another pair.

BFS

Record the first three selections and frontier states.

UCS

Explain how G changes from cost 10 to cost 4.

DFS

Separate selection order from the returned path.

IDS

Predict the first solution and discuss cost optimality.

State the successor order and the location of the goal test before tracing.

Workshop solution: depth and cost diverge

Method	First returned path	Actions	Cost
BFS	$S \rightarrow A \rightarrow G$	2	10
UCS	$S \rightarrow B \rightarrow G$	2	4
DFS as demonstrated	$S \rightarrow A \rightarrow G$	2	10
IDS with alphabetical DLS	$S \rightarrow A \rightarrow G$	2	10

IDS finds a shallowest goal here. Unequal costs make that insufficient for minimum cost.

Six checks before implementing

01

State or node: which has a parent?

The node. State equality describes the configuration, not its path.

02

When does BFS minimize cost?

Equal positive step costs turn minimum depth into minimum cost.

03

When can UCS accept a goal?

When a non-stale goal is selected with the lowest path cost.

04

Why improve a known state in UCS?

Its first discovered path may be more expensive.

05

What does CUTOFF mean?

The depth bound stopped a branch that could continue.

06

What does IDS repeat?

Fresh depth-limited searches with progressively larger limits.

One model, several ways to explore it

01	02	03	04
Model	→ Order	→ Control	→ Verify
State and legal transitions	Frontier strategy	Duplicates and bounds	Goal, cost and resources
A solution	A valid path to a goal.		
An optimal solution	A valid path with minimum cost under the chosen cost model.		
An honest result	A solution, proven finite exhaustion, or an explicitly reported limit.		

Good search begins with a correct abstraction and ends with a justified claim.

Sources and further reading

Supplied course presentation

01_Presentacion_Tema2_Problem_Solving_Search(3).pptx, 44 slides. Main source for scope, examples and lab terminology.

Russell and Norvig · Artificial Intelligence: A Modern Approach

Chapter 3: problem formulation and uninformed search.

Official textbook resources

UC Berkeley · CS188

State spaces, search problems and uninformed strategies.

Uninformed Search

Examples in this presentation

The six-state graph, numerical traces, simulators and exercises are teaching constructions. Romania distances follow the textbook example.

What if the agent knew where to look?

Uninformed search orders candidates using depth or accumulated cost.

Unit 3 adds a heuristic

$h(n)$

An estimate of the remaining cost.

A question to carry forward

When does that estimate speed up search without losing the guarantee we need?

The state model, legal transitions, path cost and goal test remain essential.